



SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

OPERATING SYSTEMS LABORATORY

Course Code: CSE309R01

Semester: V

Lab Manual

2026

ROHITH KANNA GV

128003214

A-44

**SCHOOL OF COMPUTING
SHANMUGHA ARTS, SCIENCE, TECHNOLOGY AND RESEARCH
ACADEMY
(SASTRA Deemed to be University)
Tirumalaisamudram, Thanjavur-613 401**

Exercise No. 7. Simulate Peterson's Algorithm

Write a program to implement Peterson algorithm for

synchronization Objectives

To Simulate Peterson's algorithms for mutual exclusion for two process and then to extend to n process

Prerequisite / Tools Required (optional)

Knowledge of critical-sections, mutual exclusion, bounded waiting, Synchronization and producer-consumer problem

Procedure / Algorithm

- ☐ Create user level threads using pthreads
- ☐ Develop a function that access a shared resource through Peterson
- ☐ Associate that function to two threads
- ☐ Implement Peterson logic for mutual exclusion

Sample Input

Output

For 2 process-

```
#include <stdio.h>
#include <pthread.h>
```

```
int flag[2] = {0, 0};
int turn;
int shared = 0;
```

```
void enter(int i)
{
    int j = 1 - i;
```

```

    flag[i] = 1;
    turn = j;

    while(flag[j] && turn == j);
}

void leave(int i)
{
    flag[i] = 0;
}

void *process(void *arg)
{
    int i = *(int *)arg;
    int j;

    for(j = 0; j < 3; j++)
    {
        enter(i);

        printf("Process %d entered critical section\n", i);
        shared++;
        printf("Shared value = %d\n", shared);

        leave(i);

        printf("Process %d left critical section\n\n", i);
    }

    return NULL;
}

int main()
{
    pthread_t t1, t2;
    int p1 = 0, p2 = 1;

    pthread_create(&t1, NULL, process, &p1);
    pthread_create(&t2, NULL, process, &p2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final shared value = %d\n", shared);

    return 0;
}

```

}

```
guest-z9znvg@admin123-VirtualBox:~$ gedit peterson.c
guest-z9znvg@admin123-VirtualBox:~$ gcc -o pete1 peterson.c -pthread
guest-z9znvg@admin123-VirtualBox:~$ ./pete1
The Process 0 is waiting to enter critical section
The process 0 has entered the critical section
Process Id:0 Counter:1
The Process 1 is waiting to enter critical section
Process 0 has exited the critical Section
The process 1 has entered the critical section
Process Id:1 Counter:2
The Process 0 is waiting to enter critical section
Process 1 has exited the critical Section
The process 0 has entered the critical section
Process Id:0 Counter:3
The Process 1 is waiting to enter critical section
Process 0 has exited the critical Section
The process 1 has entered the critical section
Process Id:1 Counter:4
The Process 0 is waiting to enter critical section
Process 1 has exited the critical Section
The process 0 has entered the critical section
Process Id:0 Counter:5
The Process 1 is waiting to enter critical section
Process 0 has exited the critical Section
The process 1 has entered the critical section
Process Id:1 Counter:6
The Process 0 is waiting to enter critical section
Process 1 has exited the critical Section
The process 0 has entered the critical section
Process Id:0 Counter:7
The Process 1 is waiting to enter critical section
Process 0 has exited the critical Section
the process 1 has entered the critical section
```

```
Process Id:0 Counter:5
The Process 1 is waiting to enter critical section
Process 0 has exited the critical Section
The process 1 has entered the critical section
Process Id:1 Counter:6
The Process 0 is waiting to enter critical section
Process 1 has exited the critical Section
The process 0 has entered the critical section
Process Id:0 Counter:7
The Process 1 is waiting to enter critical section
Process 0 has exited the critical Section
The process 1 has entered the critical section
Process Id:1 Counter:8
The Process 0 is waiting to enter critical section
Process 1 has exited the critical Section
The process 0 has entered the critical section
Process Id:0 Counter:9
The Process 1 is waiting to enter critical section
Process 0 has exited the critical Section
The process 1 has entered the critical section
Process Id:1 Counter:10
Process 1 has exited the critical Section
Final shared counter value:10
```

For n process-

```

#include <stdio.h>
#include <pthread.h>

#define MAX 10

int n;
int flag[MAX];
int turn[MAX];
int shared = 0;

void lock(int i)
{
    int j;

    flag[i] = 1;
    turn[i] = 1;

    for(j = 0; j < n; j++)
    {
        if(j != i)
        {
            turn[j] = i;
            while(flag[j] && turn[j] == i);
        }
    }
}

void unlock(int i)
{
    flag[i] = 0;
}

void *process(void *arg)
{
    int i = *(int *)arg;
    int k;

    for(k = 0; k < 3; k++)
    {
        lock(i);

        printf("Process %d entered critical section\n", i);
        shared++;
        printf("Process %d: Shared value = %d\n", i, shared);
        printf("Process %d left critical section\n\n", i);

        unlock(i);
    }
}

```

```

    }

    return NULL;
}

int main()
{
    pthread_t threads[MAX];
    int id[MAX];
    int i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        flag[i] = 0;
        turn[i] = 0;
        id[i] = i;

        pthread_create(&threads[i], NULL, process, &id[i]);
    }

    for(i = 0; i < n; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Final shared value = %d\n", shared);

    return 0;
}

```

```
guest-z9znvg@admin123-VirtualBox:~$ ./pete2.out
Enter the number of process (2-10):3
The Process 0 is waiting to enter critical section
The process 0 has entered the critical section
Process Id:0 Counter:1
The Process 1 is waiting to enter critical section
The Process 2 is waiting to enter critical section
Process 0 has exited the critical Section
The process 2 has entered the critical section
Process Id:2 Counter:2
The Process 0 is waiting to enter critical section
Process 2 has exited the critical Section
The process 1 has entered the critical section
Process Id:1 Counter:3
The Process 2 is waiting to enter critical section
Process 1 has exited the critical Section
The process 0 has entered the critical section
Process Id:0 Counter:4
The Process 1 is waiting to enter critical section
Process 0 has exited the critical Section
The process 2 has entered the critical section
Process Id:2 Counter:5
Process 2 has exited the critical Section
The process 1 has entered the critical section
Process Id:1 Counter:6
Process 1 has exited the critical Section
Final shared counter value:6
```

Observation

Note: Compilation command: gcc -o thread_program.c -pthread

Write syntax for pthread_create(), pthread_join(), and other thread-related functions from the POSIX Threads (pthread) library.

Extend Peterson solution for 'n' process from '2' process. Write syntax of the system calls used to implement the solution.

Syntax of pthread_create():

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,  
                  void *(*start_routine)(void *), void *arg);
```

Creates a new thread and executes the specified function.

Syntax of pthread_join():

```
int pthread_join(pthread_t thread, void **retval);
```

Waits for the specified thread to finish execution.

pthread_exit():

```
void pthread_exit(void *retval);
```

Terminates the calling thread.

pthread_self():

```
pthread_t pthread_self(void);
```

Returns the ID of the calling thread.

pthread_equal():

```
int pthread_equal(pthread_t t1, pthread_t t2);
```

Compares two thread IDs.

Result

Thus, Peterson's algorithm was successfully implemented using pthreads and mutual exclusion was achieved while accessing the shared resource.